BÀI TẬP CSP VÀ GAME TRONG AI

Họ và tên	Trần Đình Thi
MSSV / Lớp	3123410348 Lớp DCT1236

Mục tiêu học tập

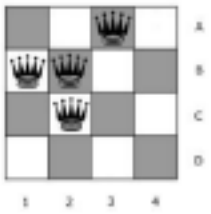
• Mô hình hóa và giải các bài toán thỏa mãn ràng buộc (CSP).
• Vận dụng Forward Checking, AC-3, MRV và LCV trong CSP.
• Phân tích Minimax, Alpha-Beta Pruning và Expectimax trong trò chơi đối kháng.
• Nhận biết ảnh hưởng của thứ tự duyệt và cắt tỉa đến hiệu quả tìm kiếm.
• Vận dụng các kỹ thuật trên vào bài toán 4-Queens, điều phối đường băng và Magic Square.

Nội dung	Kỹ thuật	Ý nghĩa chính
CSP	Forward Checking	Loại giá trị xung đột sau mỗi phép gán
CSP	AC-3	Duy trì tính nhất quán cung
CSP	MRV / LCV	Chọn biến khó trước / chọn giá trị ít ràng buộc nhất
Game	Minimax / Alpha-Beta	Tìm quyết định tối ưu và cắt tỉa nhánh không ảnh hưởng
Game	Expectimax	Mô hình hóa nút ngẫu nhiên bằng kỳ vọng

Bài 1. Bài toán 4-Queens và Hill Climbing

Hình dưới trình bày giả mã thuật toán hill-climbing cho bài toán 4 quân hậu và một trạng thái khởi đầu. Các quân hậu di chuyển như trong cờ vua: có thể đi nhiều ô theo hàng, cột hoặc đường chéo miễn là đường đi không bị chặn.

```
function 4-QUEENS-HILL-CLIMBING(problem)
    current_state = problem.INITIAL-STATE
    loop do
        neighbor = state with lowest number of conflicts between queens obtained by
                   making one legal chess move from current_state
        if neighbor.num_conflicts >= current_state.num_conflicts
            return current_state
        current_state = neighbor
```



Hình 1. Mã giả Hill Climbing và trạng thái khởi đầu của bài toán 4-Queens

a) Hãy ghi trạng thái của từng quân hậu sau một vòng lặp của thuật toán. Một vòng lặp tương đương với việc di chuyển một quân hậu.

Ở trạng thái ban đầu (**A3, B1, B2, C2**), có **4 cặp xung đột** trực tiếp. Sau khi di chuyển hợp lệ quân hậu từ **B2 đến D4**, số xung đột giảm xuống mức thấp nhất là **1 xung đột** (chỉ còn cặp **B1 - C2**).

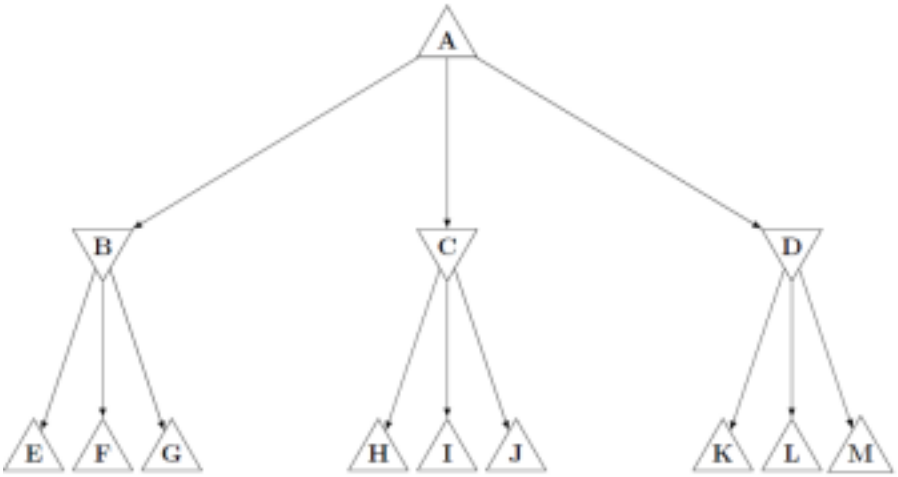
b) Thuật toán hill-climbing ở trên có dừng với trạng thái khởi đầu đã cho hay không? Nếu có, trạng thái cuối thuộc loại nào? Chọn tất cả phương án phù hợp.

- Thuật toán không dừng ngay ở trạng thái ban đầu mà sẽ tiếp tục di chuyển sang trạng thái láng giềng có $h = 2$.
- Sau một số vòng lặp tiếp theo, thuật toán sẽ bị kẹt tại một trạng thái có $h > 0$ mà mọi láng giềng xung quanh đều không thể giảm thêm xung đột. Vì không thể đạt tới $h = 0$ (nghiệm toàn cục), trạng thái cuối cùng dừng lại thuộc loại Cực trị cục bộ.

Đáp án câu b: **B) Cực trị cục bộ (local optimum)**

Bài 2. Alpha-Beta Pruning kết hợp Iterative Deepening

Xét cây trò chơi trong Hình 2. Gốc A là nút MAX; B, C, D là các nút MIN. Bảng bên phải cho giá trị hàm đánh giá f(s) của các trạng thái. Iterative deepening có thể kết hợp với cắt tỉa để tăng khả năng pruning và giảm thời gian tìm kiếm.



Evaluation Function	
State, s	$f(s)$
B	4
C	14
D	19
E	5
F	7
G	8
H	17
I	16
J	15
K	23
L	20
M	26

Hình 2. Cây trò chơi và bảng giá trị hàm đánh giá

Thực hiện theo hai bước: (1) sắp thứ tự các cây con B, C, D dựa trên f(B), f(C), f(D), từ tốt nhất đến xấu nhất theo góc nhìn của nút cha; (2) áp dụng Alpha-Beta Pruning theo thứ tự mới, giới hạn độ sâu đến các nút cháu E, F, G, ..., và dùng giá trị đánh giá tại các nút cháu.

a) Thứ tự các cây con sau khi sắp xếp là:

Vì nút cha A chọn giá trị lớn nhất, góc nhìn tốt nhất của góc A đối với cây con B,C,D là cây có giá trị f(s) cao nhất

=> Chọn phương án B->C->D

b) Với thứ tự mới, hãy liệt kê các nút KHÔNG được thăm do bị cắt tỉa. Các nút cháu trong mỗi cây con vẫn được xét từ trái sang phải.

Nhánh 1: Nút D (nút MIN)

- Nhận $(\alpha = -\infty, \beta = +\infty)$ từ A.
- Các con của D gồm: K, L, M.
  - Thăm K: $f(K) = 23 \implies \beta = \min(+\infty, 23) = 23$.
  - Thăm L: $f(L) = 20 \implies \beta = \min(23, 20) = 20$.
  - Thăm M: $f(M) = 26 \implies \beta = \min(20, 26) = 20$.
- Giá trị trả về từ D là: $v(D) = 20$.
- Cập nhật tại A:
  - $\alpha = \max(\alpha, 20) = \max(-\infty, 20) = 20$
  - $\beta = +\infty$

Nhánh 2: Nút C (nút MIN)

- Nhận $(\alpha = 20, \beta = +\infty)$ từ A.
- Các con của C gồm: H, I, J.
  - Thăm H: $f(H) = 17 \implies \beta = \min(+\infty, 17) = 17$.
  - Lúc này ta có:
    - $\beta = 17 \leq \alpha = 20$
  - Điều kiện cắt tỉa α-β xảy ra ($\alpha \geq \beta$).
  - **Cắt tỉa toàn bộ các nhánh còn lại của** C: Cắt tỉa I và J.
- Giá trị trả về từ $C \leq 17$ (nhỏ hơn $\alpha = 20$), nên A giữ nguyên $\alpha = 20$.

Nhánh 3: Nút B (nút MIN)

- Nhận $(\alpha = 20, \beta = +\infty)$ từ A.
- Các con của B gồm: E, F, G.
  - Thăm E: $f(E) = 5 \implies \beta = \min(+\infty, 5) = 5$.

- Lúc này ta có:

$$\beta = 5 \leq \alpha = 20$$

- Điều kiện cắt tỉa α-β xảy ra ($\alpha \geq \beta$).
- **Cắt tỉa toàn bộ các nhánh còn lại của** B: Cắt tỉa F và G.

- Giá trị trả về từ $B \leq 5$, không ảnh hưởng tới A.

Các nút KHÔNG được thăm do bị cắt tỉa là: I, J, F, G

c) Alpha-Beta Pruning chuẩn đảm bảo cho cùng nước đi với Minimax đầy đủ. Phương pháp iterative deepening kết hợp pruning như mô tả ở trên có luôn đảm bảo trả về cùng nước đi với Minimax trên toàn bộ cây hay không? Giải thích ngắn gọn.

Không

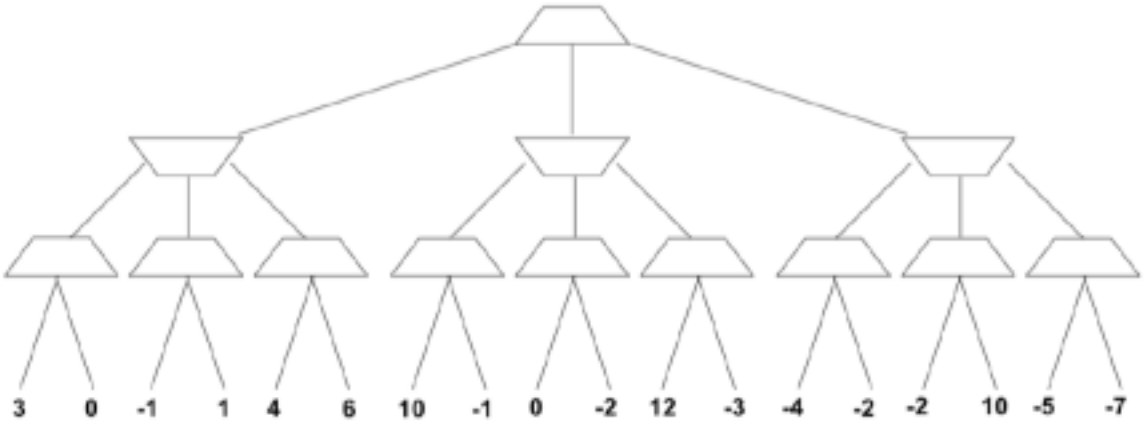
Giải thích vì sao không :

Bản chất của Alpha-Beta: Thuật toán chỉ bảo toàn kết quả khi áp dụng giá trị thực tế của cây trò chơi; việc sắp xếp lại thứ tự xét duyệt (move ordering) chỉ giúp cắt tỉa sớm hơn chứ không làm sai lệch giá trị Minimax nếu xét cùng một độ sâu.

Sai lệch do hàm đánh giá $f(s)$ **tại độ sâu bị giới hạn:** Ở đây, thuật toán dừng ở độ sâu nông (các nút cháu) và dùng **hàm heuristic** $f(s)$ thay vì giá trị Minimax chân thực tới các trạng thái kết thúc (terminal states). Do $f(s)$ chỉ là giá trị ước lượng không hoàn hảo, nó có thể dẫn đến hiện tượng chân trời (horizon effect) hoặc đánh giá sai nước đi tối ưu thực sự của toàn bộ cây trò chơi sâu hơn.

Bài 3. Adversarial Search - Alpha-Beta Pruning

Xét cây trò chơi dưới đây, trong đó nút gốc là MAX. Áp dụng Alpha-Beta Pruning và duyệt các nút kế tiếp từ trái sang phải.



Hình 3. Cây trò chơi dùng cho Alpha-Beta Pruning

Yêu cầu:

• Ghi giá trị α và β tại mỗi nút trong quá trình duyệt.

• Ghi giá trị được trả về tại mỗi nút trung gian.

• Đánh dấu X lên các cạnh hoặc nhánh bị cắt tỉa.

• Cho biết giá trị Minimax tại nút gốc và nước đi tối ưu của MAX.

Bài làm

1. NHÁNH TRÁI (Tầng 1 - Nút MIN 1.1)

Khởi tạo tại MIN 1.1: $\alpha = -\infty, \beta = +\infty$.

- Nút MAX 2.1:
  - Nhận $\alpha = -\infty, \beta = +\infty$.
  - Thăm lá 3 $\implies \alpha = \max(-\infty, 3) = 3$.
  - Thăm lá 0 $\implies \alpha = \max(3, 0) = 3$.
  - Giá trị trả về nút MAX 2.1: $v = 3$.
  - Cập nhật MIN 1.1: $\beta = \min(+\infty, 3) = 3$. Hiện tại: $[\alpha = -\infty, \beta = 3]$.
- Nút MAX 2.2:
  - Nhận $\alpha = -\infty, \beta = 3$.
  - Thăm lá -1 $\implies \alpha = \max(-\infty, -1) = -1$.
  - Thăm lá 1 $\implies \alpha = \max(-1, 1) = 1$.
  - Giá trị trả về nút MAX 2.2: $v = 1$.
  - Cập nhật MIN 1.1: $\beta = \min(3, 1) = 1$. Hiện tại: $[\alpha = -\infty, \beta = 1]$.
- Nút MAX 2.3:
  - Nhận $\alpha = -\infty, \beta = 1$.
  - Thăm lá 4 $\implies \alpha = \max(-\infty, 4) = 4$.
  - Lúc này: $\alpha = 4 \geq \beta = 1 \rightarrow$ CẮT TỈA (Prune).
  - Nhánh bị cắt: Nhánh dẫn đến lá 6 (đánh dấu X lên cạnh nối tới lá 6).

- Giá trị trả về nút MAX 2.3: $v \geq 4$.
- Cập nhật MIN 1.1: $\beta = \min(1, 4) = 1$.

⇒ Giá trị trả về tại Nút MIN 1.1 (Nhánh Trái): $v = 1$.

⇒ Cập nhật tại Nút Gốc (MAX): $\alpha = \max(-\infty, 1) = 1, \beta = +\infty$.

2. NHÁNH GIỮA (Tầng 1 - Nút MIN 1.2)

Nhận từ Gốc: $\alpha = 1, \beta = +\infty$.

- Nút MAX 2.4:
  - Nhận $\alpha = 1, \beta = +\infty$.
  - Thăm lá 10 $\implies \alpha = \max(1, 10) = 10$.
  - Thăm lá -1 $\implies \alpha = \max(10, -1) = 10$.
  - Giá trị trả về nút MAX 2.4: $v = 10$.
  - Cập nhật MIN 1.2: $\beta = \min(+\infty, 10) = 10$. Hiện tại: $[\alpha = 1, \beta = 10]$.
- Nút MAX 2.5:
  - Nhận $\alpha = 1, \beta = 10$.
  - Thăm lá 0 $\implies \alpha = \max(1, 0) = 1$.
  - Thăm lá -2 $\implies \alpha = \max(1, -2) = 1$.
  - Giá trị trả về nút MAX 2.5: $v = 0$ (vì nút MAX chọn $\max(0, -2) = 0$).
  - Cập nhật MIN 1.2: $\beta = \min(10, 0) = 0$.
  - Lúc này tại MIN 1.2: $\beta = 0 \leq \alpha = 1 \rightarrow$ CẮT TỈA (Prune).
  - Nhánh bị cắt: Toàn bộ nhánh con thứ ba của MIN 1.2 là Nút MAX 2.6 (gồm cả hai lá 12 và -3). Đánh dấu X lên cạnh nối từ MIN 1.2 xuống MAX 2.6.

⇒ Giá trị trả về tại Nút MIN 1.2 (Nhánh Giữa): $v \leq 0$ (không vượt qua $\alpha = 1$, nên Gốc giữ nguyên $\alpha = 1$).

3. NHÁNH PHẢI (Tầng 1 - Nút MIN 1.3)

Nhận từ Gốc: $\alpha = 1, \beta = +\infty$.

- Nút MAX 2.7:
  - Nhận $\alpha = 1, \beta = +\infty$.
  - Thăm lá -4 $\implies \alpha = \max(1, -4) = 1$.
  - Thăm lá -2 $\implies \alpha = \max(1, -2) = 1$.

- Giá trị trả về nút MAX 2.7: $v = -2$ (vì $\max(-4, -2) = -2$).
- Cập nhật MIN 1.3: $\beta = \min(+\infty, -2) = -2$.
- Lúc này tại MIN 1.3: $\beta = -2 \leq \alpha = 1 \rightarrow$ CẮT TỈA (Prune).
- Nhánh bị cắt: Toàn bộ nhánh con thứ hai của MIN 1.3 là Nút MAX 2.8 (gồm tất cả các lá -2, 10, -5, -7). Đánh dấu X lên cạnh nối từ MIN 1.3 xuống MAX 2.8.

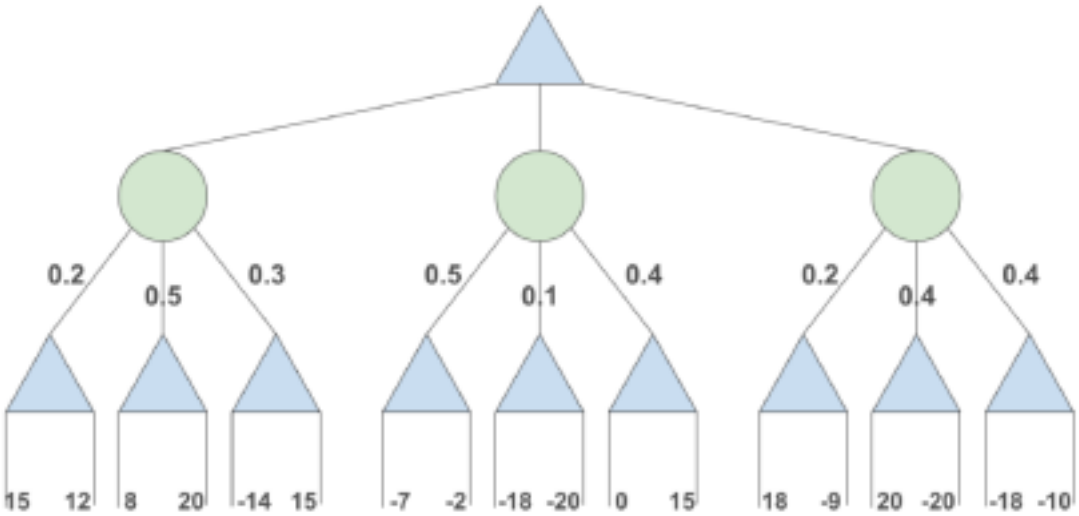
⇒ Giá trị trả về tại Nút MIN 1.3 (Nhánh Phải): $v \leq -2$.

Kết quả yêu cầu:

- Giá trị Minimax tại nút gốc: 0
- Nước đi tối ưu của MAX: Chọn nhánh giữa (Nút 2).
- Các nhánh bị cắt tỉa (đánh dấu X): Nhánh chứa các lá từ -2 đến -7 ở góc bên phải (Sub-node 3.3).

Bài 4. Expectimax và Chance Nodes

Xét cây trò chơi trong Hình 4. Nút gốc là MAX (tam giác màu xanh), các nút tròn màu xanh lá là chance node. Xác suất của mỗi nhánh được ghi trên cạnh. Các nút MAX ở tầng dưới chọn giá trị lớn nhất trong các lá của chúng.



Hình 4. Cây Expectimax với chance nodes

a) Duyệt các nút con từ trái sang phải. Hãy tính giá trị được trả về tại từng nút MAX, từng chance node và tại nút gốc.

1. Giá trị tại các nút MAX tầng dưới cùng (chọn giá trị lớn nhất trong các lá)

- **Nhánh con thuộc Chance Node 1 (trái)**:
  - Nút MAX 1: $\max(15, 12) = 15$

- Nút MAX 2: $\max(8, 20) = 20$
- Nút MAX 3: $\max(-14, 15) = 15$

- **Nhánh con thuộc Chance Node 2 (giữa)**:
  - Nút MAX 4: $\max(-7, -2) = -2$

    Nút MAX 5: $\max(-18, -20) = -18$
  - Nút MAX 6: $\max(0, 15) = 15$

    Nhánh con thuộc Chance Node 3 (phải):
  - Nút MAX 7: $\max(18, -9) = 18$
  - Nút MAX 8: $\max(20, -20) = 20$

    Nút MAX 9: $\max(-18, -10) = -10$

2. Giá trị tại từng Chance Node (tính giá trị kỳ vọng / trung bình có trọng số theo xác suất):

- **Chance Node 1 (Trái)** với các xác suất tương ứng $0.2, 0.5, 0.3$:
  - $V(C_1) = 0.2 \times 15 + 0.5 \times 20 + 0.3 \times 15 = 3 + 10 + 4.5 = 17.5$
- **Chance Node 2 (Giữa)** với các xác suất tương ứng $0.5, 0.1, 0.4$:
  - $V(C_2) = 0.5 \times (-2) + 0.1 \times (-18) + 0.4 \times 15 = -1 - 1.8 + 6 = 3.2$
- **Chance Node 3 (Phải)** với các xác suất tương ứng $0.2, 0.4, 0.4$:
  - $V(C_3) = 0.2 \times 18 + 0.4 \times 20 + 0.4 \times (-10) = 3.6 + 8 - 4 = 7.6$

3. Giá trị tại nút Gốc (nút MAX ở đỉnh)

$$V(Gốc) = \max(V(C_1), V(C_2), V(C_3)) = \max(17.5, 3.2, 7.6) = 17.5$$

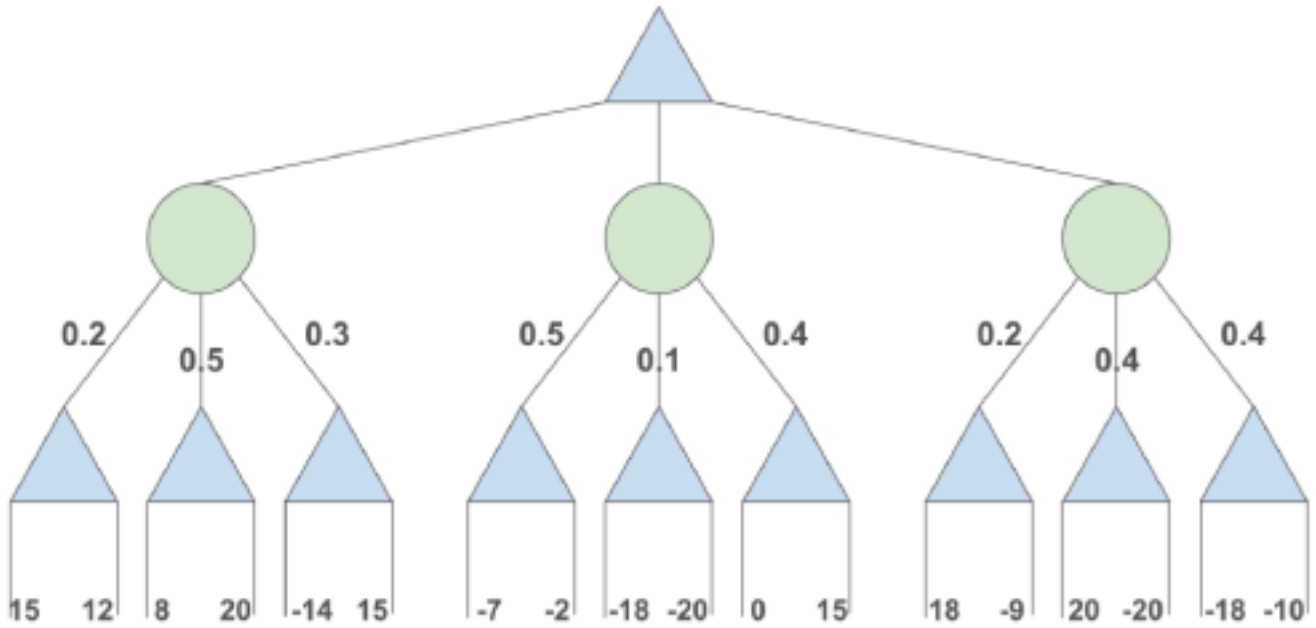
b) Nếu chưa biết thêm giới hạn nào về giá trị lá, những nút nào trong cây trên có thể được cắt tỉa mà vẫn đảm bảo
kết quả chính xác? Giải thích.

Không có nút nào có thể được cắt tỉa.

Giải thích:

Trong cây Expectimax, giá trị của một nút ngẫu nhiên (chance node) là trung bình có trọng số của tất cả các nhánh con của nó.
Nếu không biết trước giới hạn trên và giới hạn dưới (bounds) của các giá trị lá, một nút lá chưa được xét hoàn toàn có thể mang một giá trị cực lớn ($+\infty$) hoặc cực nhỏ ($-\infty$), từ đó làm thay đổi hoàn toàn giá trị kỳ vọng của nút chance node và làm thay đổi quyết định tại nút gốc. Do đó, thuật toán buộc phải thăm đầy đủ tất cả các nút lá.

c) Giả sử mỗi nút lá đều có giá trị nằm trong đoạn [-20, 20]. Hãy giải lại cây và cắt tỉa các nhánh không cần thiết;
vẫn duyệt từ trái sang phải.



Hình 5. Cây Expectimax dùng cho câu 4(c)

Chance node trái: Duyệt hết do chưa có mốc so sánh.

- Giá trị: $V(C_1) = 0.2 \times 15 + 0.5 \times 20 + 0.3 \times 15 = 17.5$ $\implies$ Nút gốc lưu giá trị tạm thời $v^* = 17.5$.

Chance node giữa (Cắt tỉa):

- Nhánh con 1 có giá trị $\max(-7, -2) = -2$.
- Cận trên lớn nhất của nút giữa nếu 2 nhánh còn lại đạt tối đa (20):
    - $V_{\max}(C_2) = 0.5 \times (-2) + 0.1 \times 20 + 0.4 \times 20 = 9 < 17.5$
- Cắt tỉa: Toàn bộ nhánh con thứ hai (chứa $[-18, -20]$) và nhánh con thứ ba (chứa $[0, 15]$)

Chance node phải:

- Nhánh con 1: $\max(18, -9) = 18$.
- Nhánh con 2: Thăm lá đầu tiên thấy đạt giá trị tối đa là $20 = V_{\max}$ => Cắt tỉa lá $[-20]$ ngay sau đó.
- Nhánh con 3: Duyệt cả hai lá được $\max(-18, -10) = -10$.
- Giá trị: $V(C_3) = 0.2 \times 18 + 0.4 \times 20 + 0.4 \times (-10) = 7.6$.

Các nút/nhánh bị cắt tỉa: *Nút MAX 5 và Nút MAX 6 (gồm các lá -18,-20,0,15).*

Bài 5. Câu hỏi thảo luận về CSP

a) Forward Checking khác AC-3 ở điểm nào? So sánh phạm vi lan truyền ràng buộc, thời điểm thực hiện và chi phí tính toán.

Tiêu chí	Forward Checking (FC)	AC-3 (Arc Consistency 3)
Bản chất khác biệt	Chỉ kiểm tra tính nhất quán 1 bước từ biến vừa gán sang các biến láng giềng.	Lan truyền ràng buộc liên tục qua toàn bộ mạng lưới cho đến khi đạt trạng thái ổn định.
Phạm vi lan truyền	**Cục bộ (1 bước):** Chỉ lan truyền từ biến hiện tại tới các biến chưa gán kề nó; không phát hiện được xung đột gián tiếp giữa các biến chưa gán.	**Toàn cục (nhiều bước):** Khi miền giá trị của một biến bị thu hẹp, tiếp tục lan truyền sang các biến láng giềng khác cho đến khi không còn giá trị nào vi phạm.
Thời điểm thực hiện	Thực hiện ngay sau mỗi bước gán giá trị cho một biến.	Có thể dùng làm bước tiền xử lý (trước khi tìm kiếm) hoặc thực hiện sau mỗi bước gán (MAC - Maintaining Arc Consistency).
Chi phí tính toá	**Thấp:** Đơn giản, tính toán nhanh ở mỗi bước nhưng cắt tỉa ít không gian trạng thái hơn.	**Cao hơn nhiều:** Độ phức tạp thời gian $O(c \cdot d^3)$ (với c ràng buộc, d kích thước miền giá trị), đổi lại phát hiện ngõ cụt sớm hơn rất nhiều.

b) Vì sao các heuristic sau thường được dùng trong bài toán CSP?

i) Minimum Remaining Values (MRV)

Nguyên lý Fail-First: Biến có miền giá trị hẹp nhất là biến dễ gây xung đột nhất. Thử biến này trước giúp phát hiện sớm nhánh cụt để quay lui ngay lập tức, tránh lãng phí công sức duyệt sâu.

ii) Least Constraining Value (LCV)

Nguyên lý Fail-Last: Chọn giá trị nào loại bỏ ít lựa chọn nhất của các biến láng giềng chưa gán.

Tối đa hóa khả năng tìm thấy lời giải hợp lệ ngay ở nhánh đầu tiên mà không cần phải quay lui (backtrack).

Bài 6. CSP - Điều phối không lưu

Có năm máy bay A, B, C, D và E cùng hai đường băng: quốc tế và nội địa. Cần xếp cho mỗi máy bay một khung giờ và một đường băng để hạ cánh hoặc cất cánh. Mỗi đường băng có bốn khung giờ 1, 2, 3, 4. Bài toán phải thỏa các ràng buộc sau:

• Máy bay B bị hỏng một động cơ và phải hạ cánh ở khung giờ 1.

• Máy bay D chỉ có thể đến sân bay để hạ cánh từ khung giờ 3 trở đi.

• Máy bay A sắp hết nhiên liệu và chỉ có thể chờ tối đa đến khung giờ 2.

• Máy bay D phải hạ cánh trước khi máy bay C cất cánh vì có hành khách nối chuyến từ D sang C. • Không có hai máy bay nào được đặt cùng một khung giờ trên cùng một đường băng.

a) Hãy mô hình hóa bài toán dưới dạng CSP: xác định biến, miền giá trị và các ràng buộc đơn/nguyên (unary) và nhị phân (binary). Viết ràng buộc bằng ký hiệu toán học hoặc logic.

Biến: $X = \{A, B, C, D, E\}$

Miền giá trị (Domain): Với mọi $X_i \in X$:

$$D_{X_i} = \{1, 2, 3, 4\} \times \{Quốctế, Nộiđịa\}$$

Ký hiệu $T(X_i) \in \{1, 2, 3, 4\}$ là khung giờ và $R(X_i) \in \{Quốctế, Nộiđịa\}$ là đường băng.

Ràng buộc đơn (Unary constraints):

- $T(B) = 1$

- $T(D) \geq 3$
- $T(A) \leq 2$

Ràng buộc nhị phân (Binary constraints):

- Hành khách nối chuyến từ D sang C:

  - $T(D) < T(C)$

- Không dùng chung khung giờ trên cùng đường băng (với mọi cặp X_i, X_j phân biệt):
  $\neg(Time(X_i) = Time(X_j) \wedge Runway(X_i) = Runway(X_j))$

Bổ sung hai ràng buộc sau:

• A, B, C là các chuyến bay quốc tế và chỉ dùng đường băng quốc tế.

• D, E là các chuyến bay nội địa và chỉ dùng đường băng nội địa.

b) Với hai ràng buộc bổ sung, loại đường băng không cần nằm trong miền giá trị vì đã được xác định bởi biến. Hãy xác định miền mới và vẽ đồ thị ràng buộc của bài toán.

Miền giá trị mới:

$$D_A = D_B = D_C = D_D = D_E = \{1, 2, 3, 4\}$$

Đồ thị ràng buộc (Constraint Graph):

- **Các đỉnh:** A, B, C, D, E
- **Các cạnh (ràng buộc nhị phân):**
  - Cạnh giữa các cặp cùng đường băng (ràng buộc $\neq$):
    - Giữa nhóm Quốc tế: $(A - B)$, $(A - C)$, $(B - C)$
    - Giữa nhóm Nội địa: $(D - E)$
  - Cạnh liên nhóm nối chuyến: $(D - C)$ (ràng buộc $T(D) < T(C)$)
- *(Biểu diễn liên kết: Tam giác $A - B - C$, cạnh $D - E$, và cầu nối có hướng/ràng buộc $D \rightarrow C$).*

c) Sau khi áp dụng arc consistency, miền của các biến còn lại là gì? Bắt đầu bằng việc áp dụng các ràng buộcunary. Hãy gạch bỏ những giá trị bị loại khỏi miền.

Bước 1: Áp dụng các ràng buộc đơn (Node Consistency):

- $D_B = \{1\}$ (loại $2, 3, 4$)
- $D_D = \{3, 4\}$ (loại $1, 2$)
- $D_A = \{1, 2\}$ (loại $3, 4$)

- $D_C = \{1, 2, 3, 4\}$
- $D_E = \{1, 2, 3, 4\}$

Bước 2: Lan truyền ràng buộc nhị phân (Arc Consistency):

Xét cung (A, B) với A != B: Vì D_B = {1} => gạch bỏ 1 khỏi D_A
=> D_A = {2}
Xét cung (C, A) và (C, B) với C != A, C != B: Vì D_A = {2} và D_B = {1} => gạch bỏ 1, 2 khỏi D_C
=> D_C = {3, 4}
Xét cung (D, C) với T(D) < T(C): Do D_C chỉ còn {3, 4} (tối đa là 4) nên D không thể bằng 4 => gạch bỏ 4 khỏi D_D
=> D_D = {3}
Xét lại cung (C, D) với T(C) > T(D): Do D_D = {3} nên C phải lớn hơn 3 => gạch bỏ 3 khỏi D_C
=> D_C = {4}
Xét cung (E, D) với E != D: Do D_D = {3} => gạch bỏ 3 khỏi D_E
=> D_E = {1, 2, 4}

Miền giá trị còn lại của biến
D_A = {2}
D_B = {1}
D_C = {4}
D_D = {3}
D_E = {1, 2, 4}

d) Thay vì duy trì arc consistency toàn cục, hãy dùng Forward Checking kết hợp Backtracking Search. Chọn biến theo MRV; khi hòa, chọn ký tự nhỏ hơn trước; chọn giá trị nhỏ hơn trước. Hãy liệt kê các cặp (biến, giá trị) theo đúng thứ tự phát sinh, kể cả các phép gán bị hoàn tác khi gặp dead end. Áp dụng ràng buộc unary trước khi bắt đầu.

A	1	2	3	4
B	1	2	3	4
C	1	2	3	4
D	1	2	3	4
E	1	2	3	4

Dưới đây là toàn bộ lời giải đã được chuyển sang **dạng văn bản thuần (plain text)** hoàn toàn, không chứa bất kỳ ký hiệu LaTeX hay công thức đặc biệt nào để bạn có thể copy trực tiếp vào Google Docs mà không bị lỗi hiển thị:

Quy tắc và Thứ tự ưu tiên

1. Áp dụng ràng buộc unary trước khi tìm kiếm.
2. Chọn biến theo MRV (biến có ít giá trị hợp lệ còn lại nhất).
3. Khi hòa kích thước miền: Chọn biến có chữ cái nhỏ hơn trước (A trước B, C, D, E).
4. Chọn giá trị nhỏ hơn trước (1 trước 2, 3, 4).
5. Forward Checking (FC): Khi gán một biến, lập tức loại bỏ các giá trị vi phạm ở miền của các biến kề chưa gán. Nếu xuất hiện miền rỗng, kết luận dead end và quay lui (backtrack).

Quá trình thực hiện chi tiết từng bước

Bước 0: Áp dụng ràng buộc unary ban đầu

- D_A = {1, 2} (do T(A) <= 2)
- D_B = {1} (do T(B) = 1)
- D_C = {1, 2, 3, 4}
- D_D = {3, 4} (do T(D) >= 3)
- D_E = {1, 2, 3, 4}

Bước 1: Chọn biến theo MRV

- Kích thước miền: D_B có 1 giá trị; D_A có 2; D_D có 2; D_C có 4; D_E có 4.
- Chọn biến B.
- Gán: (B, 1).
- Lan truyền Forward Checking từ B:
    - Cùng đường băng quốc tế: loại 1 khỏi D_A => D_A = {2}.
    - Cùng đường băng quốc tế: loại 1 khỏi D_C => D_C = {2, 3, 4}.
    - D và E ở đường băng nội địa nên không đổi.
    - Không có miền rỗng.

Bước 2: Chọn biến tiếp theo

- Kích thước miền: D_A có 1 giá trị; D_D có 2; D_C có 3; D_E có 4.
- Chọn biến A.
- Gán: (A, 2).

- Lan truyền Forward Checking từ A:
    - Cùng đường băng quốc tế: loại 2 khỏi D_C => D_C = {3, 4}.
    - Không có miền rỗng.

Bước 3: Chọn biến tiếp theo

- Kích thước miền: D_C có 2 giá trị; D_D có 2 giá trị; D_E có 4 giá trị.
- D_C và D_D hòa kích thước (đều bằng 2). Theo luật ưu tiên chữ cái nhỏ hơn: Chọn biến C.
- Chọn giá trị nhỏ hơn trước: Thử gán (C, 3).
- Lan truyền Forward Checking từ C:
    - Ràng buộc nối chuyến: T(D) < T(C), tức là T(D) < 3.
    - Kiểm tra miền của D hiện tại là {3, 4}. Cả hai giá trị 3 và 4 đều không nhỏ hơn 3, do đó cả hai đều bị loại.
    - Miền D_D trở thành rỗng (Dead end).
- Hoàn tác: Hủy phép gán (C, 3), khôi phục lại miền D_D = {3, 4}.

Bước 4: Thử giá trị tiếp theo cho biến C

- Gán: (C, 4).
- Lan truyền Forward Checking từ C:
    - Ràng buộc nối chuyến: T(D) < T(C), tức là T(D) < 4.
    - Trong miền D_D = {3, 4}, loại giá trị 4 => D_D = {3}.
    - Không có miền rỗng.

Bước 5: Chọn biến tiếp theo

- Kích thước miền: D_D có 1 giá trị; D_E có 4 giá trị.
- Chọn biến D.
- Gán: (D, 3).
- Lan truyền Forward Checking từ D:
    - Cùng đường băng nội địa: loại 3 khỏi D_E => D_E = {1, 2, 4}.
    - Không có miền rỗng.

Bước 6: Chọn biến cuối cùng

- Chỉ còn biến E với miền {1, 2, 4}.
- Chọn giá trị nhỏ hơn trước: Gán (E, 1).
- Tất cả 5 biến đã được gán giá trị hợp lệ. Quá trình tìm kiếm thành công.

Các giá trị trong bảng theo từng biến

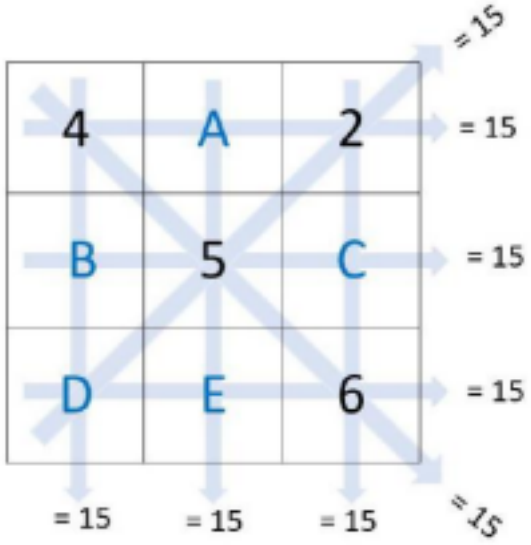
- Hàng A: Gạch 1 (do B=1), Giữ 2, Gạch 3 (do unary), Gạch 4 (do unary). Kết quả: A = 2.
- Hàng B: Giữ 1, Gạch 2 (unary), Gạch 3 (unary), Gạch 4 (unary). Kết quả: B =1
- Hàng C: Gạch 1 (do B=1), Gạch 2 (do A=2), Thử 3 (bị backtrack), Giữ 4. Kết quả: C = 4.
- Hàng D: Gạch 1 (unary), Gạch 2 (unary), Giữ 3, Gạch 4 (do điều kiện D < C). Kết quả: D = 3.
- Hàng E: Giữ 1, Không chọn 2, Gạch 3 (do D=3), Không chọn 4. Kết quả: E =1.

Thứ tự các cặp (biến, giá trị) được gán:

1. (B, 1)
2. (A, 2)
3. (C, 3)
4. (C, 4)
5. (D, 3)
6. (E, 1)

Bài 7. CSP - Magic Square

Magic Square là bảng n × n trong đó mỗi ô chứa một giá trị khác nhau thuộc {1, ..., n^2}; tổng của mỗi hàng, mỗi cột và hai đường chéo đều bằng nhau. Với bảng 3 × 3 dưới đây, tổng cần đạt là 15. Một số ô đã được cho trước, các ô ký hiệu chữ cái cần được xác định.



Hình 6. Bài toán Magic Square 3 × 3

a) Hãy mô hình hóa bài toán dưới dạng CSP: biến, miền giá trị và các ràng buộc. Có thể bỏ qua việc tạo biến cho các ô đã biết.

Tập biến:

X = {A, B, C, D, E} (tương ứng với 5 ô chưa biết giá trị trong bảng).

Miền giá trị:
Các số đã cho trước là {2, 4, 5, 6}.
Do mỗi ô nhận giá trị khác nhau thuộc {1, ..., 9}, miền giá trị của mỗi biến là các số còn lại:
D_A = D_B = D_C = D_D = D_E = {1, 3, 7, 8, 9}
Các ràng buộc:

1. Ràng buộc tổng các hàng, cột, đường chéo bằng 15:
    - Hàng 1: 4 + A + 2 = 15 => A = 9
    - Hàng 2: B + 5 + C = 15 => B + C = 10
    - Hàng 3: D + E + 6 = 15 => D + E = 9
    - Cột 1: 4 + B + D = 15 => B + D = 11
    - Cột 2: A + 5 + E = 15 => A + E = 10
    - Cột 3: 2 + C + 6 = 15 => C = 7
2. Ràng buộc phân biệt:
    - all-different(A, B, C, D, E)

b) Vẽ đồ thị ràng buộc nhị phân. Để đơn giản, có thể bỏ qua ràng buộc alldiff khi vẽ đồ thị.

Các đỉnh: Gồm 5 đỉnh biểu diễn các biến: A, B, C, D, E.
Các cạnh nối: Là các ràng buộc liên kết giữa hai biến (xuất hiện cùng nhau trong tổng của một hàng/cột):

- Cạnh (B - C): ràng buộc B + C = 10 (từ hàng 2)
- Cạnh (D - E): ràng buộc D + E = 9 (từ hàng 3)
- Cạnh (B - D): ràng buộc B + D = 11 (từ cột 1)
- Cạnh (A - E): ràng buộc A + E = 10 (từ cột 2)

c) Sử dụng đồ thị ràng buộc nhị phân và thuật toán AC-3 để thu hẹp miền, sau đó tìm một nghiệm cho Magic Square.

Bài 8. MRV và LCV trong thực hành

Ba sinh viên AJ, Sunny và Ellyse cần chọn mỗi người đúng một màu sơn sao cho không có hai người chọn cùng màu. Các tập màu có sẵn:

• AJ = {Đỏ, Vàng, Xanh lá}

• Sunny = {Đỏ, Vàng, Hồng}

• Ellyse = {Đỏ, Hồng}

a) Dùng MRV để chọn biến và LCV để chọn giá trị. Nếu hòa, phá hòa theo thứ tự cầu vồng. Hỏi thuật toán phải backtrack về phép gán trước đó bao nhiêu lần? Hãy ghi lại

thứ tự gán.

Miền giá trị ban đầu:

- AJ = {Đỏ, Vàng, Xanh lá} (kích thước = 3)
- Sunny = {Đỏ, Vàng, Hồng} (kích thước = 3)
- Ellyse = {Đỏ, Hồng} (kích thước = 2)

Bước 1: Chọn biến theo MRV

- Biến có ít giá trị nhất: **Ellyse** (kích thước = 2).
- Chọn giá trị theo LCV cho Ellyse:
  - Nếu chọn *Đỏ*: loại màu Đỏ khỏi AJ và Sunny
  - Nếu chọn *Hồng*: chỉ loại màu Hồng khỏi Sunny
  - LCV chọn giá trị ít gây hạn chế nhất: **Hồng**.
- **Gán: Ellyse = Hồng**.
- Miền còn lại:
  - AJ = {Đỏ, Vàng, Xanh lá} (kích thước = 3)
  - Sunny = {Đỏ, Vàng} (kích thước = 2)

Bước 2: Chọn biến theo MRV

- Biến có ít giá trị nhất: **Sunny** (kích thước = 2).
- Chọn giá trị theo LCV cho Sunny:
  - Cả *Đỏ* và *Vàng* đều làm giảm 1 lựa chọn của AJ
  - Phá hòa theo thứ tự cầu vồng: chọn **Đỏ**.
- **Gán: Sunny = Đỏ**.
- Miền còn lại:
  - AJ = {Vàng, Xanh lá}

Bước 3: Chọn biến cuối cùng

- Còn lại biến: **AJ**.
- Chọn theo thứ tự cầu vồng: **Vàng**.
- **Gán: AJ = Vàng**.

Kết luận

- **Số lần backtrack: 0 lần**.
- **Thứ tự gán:** (Ellyse, Hồng) -> (Sunny, Đỏ) -> (AJ, Vàng).

b) Thay MRV bằng Maximum Remaining Values và thay LCV bằng Most Constraining Value. Hỏi số lần

backtrack là bao nhiêu? So sánh với câu (a) và giải thích vì sao.

Miền giá trị ban đầu:

- AJ = {Đỏ, Vàng, Xanh lá} (kích thước = 3)
- Sunny = {Đỏ, Vàng, Hồng} (kích thước = 3)
- Ellyse = {Đỏ, Hồng} (kích thước = 2)

Bước 1: Chọn biến theo Maximum Remaining Values

- Hòa giữa AJ và Sunny (đều có kích thước = 3). Chọn AJ (hoặc Sunny, giả sử duyệt theo thứ tự xuất hiện AJ).
- Chọn giá trị theo MCV (chọn giá trị làm mất nhiều lựa chọn của các biến khác nhất):
    - *Đỏ*: loại Đỏ ở cả Sunny và Ellyse (loại 2 lựa chọn).
    - *Vàng*: chỉ loại Vàng ở Sunny (loại 1 lựa chọn).
    - *Xanh lá*: không làm mất lựa chọn của ai (loại 0 lựa chọn).
    - MCV chọn giá trị gây cản trở nhiều nhất: **Đỏ**.
- **Gán: AJ = Đỏ**.
- Miền còn lại:
    - Sunny = {Vàng, Hồng} (kích thước = 2)
    - Ellyse = {Hồng} (kích thước = 1)

Bước 2: Chọn biến theo Maximum Remaining Values

- Biến có nhiều giá trị nhất: **Sunny** (kích thước = 2).
- Chọn giá trị theo MCV cho Sunny:
    - *Vàng*: không ảnh hưởng đến Ellyse (loại 0 lựa chọn).
    - *Hồng*: loại Hồng ở Ellyse (loại 1 lựa chọn).
    - MCV chọn: **Hồng**.
- **Gán: Sunny = Hồng**.
- Miền còn lại:
    - Ellyse = {} (miền bị rỗng -> **Dead end!**).

Bước 3: Backtrack lần 1

- Hoàn tác phép gán (Sunny, Hồng).
- Thử giá trị tiếp theo cho Sunny: **(Sunny, Vàng)**.
- Miền còn lại: Ellyse = {Hồng}.

Bước 4: Chọn biến cuối cùng

- Gán: **(Ellyse, Hồng)**.
- Tìm ra lời giải: {AJ: Đỏ, Sunny: Vàng, Ellyse: Hồng}.

Kết luận

- **Số lần backtrack: 1 lần** (tại bước gán Sunny = Hồng làm Ellyse hết màu).
- **So sánh và giải thích:**
    - Ở câu a, số lần backtrack là 0, trong khi ở câu b là 1 lần.

- *Giải thích:* Nguyên lý chuẩn **MRV + LCV** tuân theo chiến lược "Fail-First" khi chọn biến (ưu tiên biến nghẽn nhất để giải quyết trước) và "Fail-Last" khi chọn giá trị (chừa lại tối đa phương án cho các biến khác), giúp thuật toán đi thẳng tới nghiệm mà không bị vướng ngõ cụt. Ngược lại, việc dùng **Maximum Remaining Values + MCV** đảo ngược cả hai nguyên lý này: chọn biến dễ nhất trước và cố tình chọn giá trị bóp nghẹt các biến còn lại nhiều nhất, dẫn đến việc đẩy các biến lân cận vào miền rỗng và buộc phải quay lui (backtrack).

--- Hết ---